Recognizing Two-Queue Graphs via Five-Copy Amplification

Research draft · awaiting human expert review
15 September 2026

Abstract

A queue layout orders the vertices of a graph and partitions its edges into classes with no nested pair. Recognizing graphs that admit one queue is NP-complete, but this alone does not establish hardness for two queues. We give a deterministic polynomial-time many-one reduction between these recognition problems. The construction joins two adjacent universal vertices to five disjoint copies of the input. Its converse follows from a positional lemma: in any two-queue layout of a two-apex join, deleting at most four original vertices leaves a one-queue order. An untouched copy therefore yields a source witness from every valid target layout. The output has $5n + 2$ vertices and $5m + 10n + 1$ edges. This supplies a corrected construction for a hardness conclusion previously claimed using a single-copy join and establishes NP-completeness of two-queue recognition.

1. Introduction

The queue number of a graph measures how many non-nesting edge classes are needed under a common vertex order. Heath and Rosenberg [1, Corollary 3.9] proved that recognizing graphs of queue number at most one is NP-complete. Allowing another queue changes the decision language: a graph rejected at threshold one may be accepted at threshold two. The known one-queue hardness result therefore requires a new reduction to establish two-queue hardness.

De Col [2, Theorem 3] proposed adding two adjacent universal vertices to the input graph. The asserted equivalence fails on a six-vertex example described in Section 5. The corresponding conference paper [3, Section 2] proves hardness results for mixed layouts, which allow both stacks and queues, and omits the pure-two-queue claim. The two-queue recognition problem remains listed in the current author-maintained agenda [6].

We retain the two-apex join and replicate the input before applying it. The new argument controls every target order, including orders in which the copies interleave. Write Q_k for the decision language of graphs admitting at most k queues.

Theorem 1. There is a deterministic polynomial-time Karp map f from Q_1 to Q_2. For an input with n vertices and m edges, its output has $5n + 2$ vertices and $5m + 10n + 1$ edges. A deterministic polynomial-time algorithm recovers a one-queue layout of the source from every valid two-queue layout of its output. Consequently, recognizing two-queue graphs is NP-complete.

The reduction uses a bound on exceptional positions rather than an assumption that an apex is extreme. At most four original vertices can obstruct the induced one-queue order. Five copies guarantee that some copy avoids all four positions. This copy is identified directly from the supplied target witness.

2. Layouts and construction

All graphs are finite, simple and undirected. A k-queue layout of a graph $G = (V, E)$ consists of a total vertex order $\prec$ and a function $c : E \to \{0, \dots, k-1\}$. For any distinct vertices ordered $u \prec x \prec y \prec v$, edges uv and xy must receive different queue indices. Juxtaposition denotes an unordered edge. Edges sharing an endpoint do not form a nested pair.

A *three-rainbow* consists of edges u_1v_1, u_2v_2, and u_3v_3 with

$$u_1 \prec u_2 \prec u_3 \prec v_3 \prec v_2 \prec v_1. \tag{1}$$

Such edges are pairwise nested and require three distinct queues. In particular, no two-queue layout contains a three-rainbow. The converse characterization for a fixed order is due to Heath and Rosenberg [1, Theorem 2.3]; the main proof below needs only this necessary condition.

For the source graph G, form five vertex-disjoint copies $G_1, \ldots, G_5$ and let X be their disjoint union. Add vertices a, b, the edge ab, and every edge from either apex to $V(X)$. If $\vee$ denotes graph join, the forward map is

$$f(G) = H = K_2 \vee X, \quad X = G_1 \cup G_2 \cup G_3 \cup G_4 \cup G_5. \tag{2}$$

Figure Figure 1 specifies its adjacency. The graph construction supplies no vertex order.

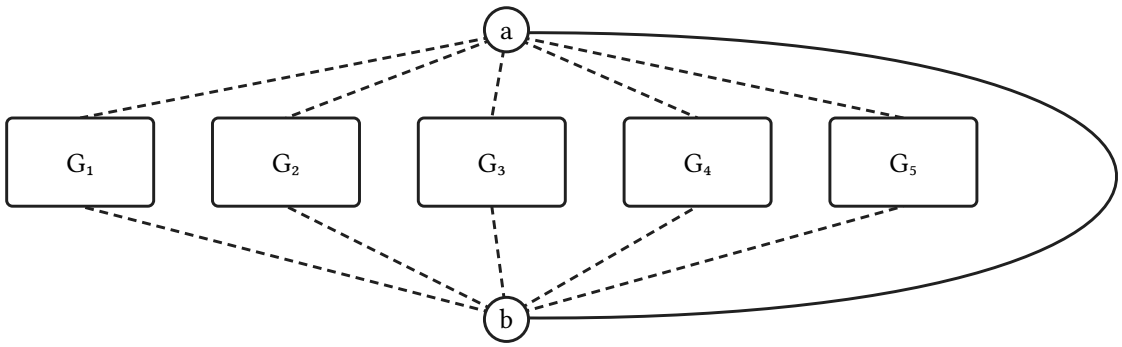


Figure 1: Five-copy construction. Each dashed bundle represents all edges between its apex and the indicated copy of G. The solid outer edge is ab. There are no edges between different copies. The drawing specifies adjacency, not a prescribed target order.

3. Four exceptional vertices

Fix any two-queue layout of $K_2 \vee X$, where X is nonempty, and name the apices so that $a \prec b$. Let ℓ and r be the first and last vertices of X. When they exist, let p be the last vertex of X before a, and let q be the first vertex of X after b. Let S contain ℓ, r and each of p, q that exists. Repeated vertices occur only once. Thus $|S| \leq 4$, including orders with no original vertex before a or after b. Figure Figure 2 illustrates the positions.



Delete ℓ, r and the existing boundary vertices p, q.

Figure 2: Exceptional positions in a supplied target order, shown when all four marked vertices exist and are distinct. Filled vertices belong to S; open vertices are the apices. Missing boundary positions are omitted from S, and coincident positions count once.

Lemma 2. The restriction of the supplied vertex order to $X - S$ has no nested edge pair. Assigning all its edges to one queue gives a one-queue layout of $X - S$.

Proof. Suppose instead that edges uv and xy survive with $u \prec x \prec y \prec v$. Their endpoints avoid S, so $\ell \prec u$ and $v \prec r$. If $v \prec b$, edges ℓb, uv, and xy form a three-rainbow. If $a \prec u$, edges ar, uv, and xy form one. Both contradict the two-queue assumption. The remaining case satisfies

$$u \prec a \prec b \prec v. \tag{3}$$

There are six possible placements of x, y relative to a, b. The following table gives a three-rainbow in each case, listing edges from outermost to innermost.

Placement	Three-rainbow	Strict endpoint order
$x \prec y \prec a$	$(\ell b, ua, xy)$	$\ell \prec u \prec x \prec y \prec a \prec b$
$b \prec x \prec y$	(ar, bv, xy)	$a \prec b \prec x \prec y \prec v \prec r$
$a \prec x \prec y \prec b$	(uv, ab, xy)	$u \prec a \prec x \prec y \prec b \prec v$
$x \prec a \prec b \prec y$	(uv, xy, ab)	$u \prec x \prec a \prec b \prec y \prec v$
$x \prec a \prec y \prec b$	(uv, xy, pa)	$u \prec x \prec p \prec a \prec y \prec v$
$a \prec x \prec b \prec y$	(uv, xy, bq)	$u \prec x \prec b \prec q \prec y \prec v$

In the fifth row, p exists because $x \prec a$. Since x avoids S, it differs from p, and the choice of p gives $x \prec p \prec a$. In the sixth row, q exists because $b \prec y$. Since y avoids S, the choice of q gives $b \prec q \prec y$. Every additional edge displayed in the table is an apex edge. All six endpoints in each row are distinct. The cases exhaust the placements of x, y, so the assumed surviving pair cannot exist. □

4. The complete reduction rule

We prove Theorem 1 by giving both witness transformations. The backward transformation uses only the source graph and the supplied target layout.

4.1. Extension of a source witness

Suppose G admits a one-queue layout with vertex order σ. Place a first, then the five copies consecutively, each with order σ, and finally b. Assign every source-copy edge to queue zero. Edges within a copy do not nest, and edges in different copies lie in disjoint order intervals.

Assign all apex edges to queue one. Two edges incident with the same apex share an endpoint. An edge incident with a and one incident with b either share an endpoint or have endpoint intervals that cross or are disjoint; they cannot nest. The edge ab shares an apex with every other edge of this queue. This is a valid two-queue layout of H.

4.2. Recovery from every target witness

For a nonempty source, label vertex v in zero-based copy i by $in + v$, where $0 \le i < 5$ and $0 \le v < n$. The apex labels are $5n$ and $5n + 1$. Given any valid target witness, locate its apices and compute S using the supplied order. At most four copies meet S. Choose the least-indexed copy that avoids S, restrict the supplied order to that copy, and replace each label $in + v$ by v. Assign every source edge to queue zero.

Lemma 2 implies that the recovered order has no nested pair. Recovery does not use the target's queue names and does not require its copies to be consecutive. If $R_k(G, w)$ denotes the k-queue witness predicate, the complete rule satisfies

$$G \in Q_1 \leftrightarrow f(G) \in Q_2, \quad R_2(f(G), w) \Longrightarrow R_1(G, g(G, w)). \tag{4}$$

For the empty source, the output is K_2, and recovery returns the empty order and edge assignment. This handles the boundary case without an exceptional-set calculation.

4.3. Encoding and resource bounds

Use adjacency-matrix encodings for the two decision languages. Malformed strings are outside both languages and map to one fixed malformed target encoding. Every well-formed source, including every NO instance, produces a simple graph. Each witness lists a vertex permutation and queue indices for the lexicographically ordered edges.

The output graph has

$$|V(H)| = 5n + 2, \quad |E(H)| = 5m + 10n + 1. \tag{5}$$

It uses $O((n+1)^2)$ matrix bits and is constructed in $O((n+1)^2)$ word operations. Selecting an intact copy and its order takes $O(n+1)$ word operations. The executable also reads the source matrix and writes its edge assignment, so these order-selection costs are not the full command costs. A conservative bound for either command on legal inputs is $O((n+1)^2 \log(n+2))$ bit operations, plus reading the actual encoded input. The recovered witness has $O(n \log(n+2) + m + 1)$ bits. Malformed inputs are rejected in polynomial encoded length.

The construction and recovery are deterministic and make no decision-oracle calls. Membership in Q_2 has a polynomial certificate: check a supplied permutation and every same-queue edge pair. Combining this NP membership with Equation 4 and the known NP-completeness of Q_1 proves Theorem 1. □

5. Relation to the single-copy claim and recent work

The two-apex construction originates in de Col's thesis [2, Theorem 3, pp. 19–20]. Its single-copy equivalence fails for the graph on vertices $0, \ldots, 5$ with edge set

$$\{(0,1),(0,2),(0,4),(1,3),(1,5),(2,3),(2,5),(3,5)\}. \tag{6}$$

An exhaustive check of all $6! = 720$ source orders finds no one-queue order. Nevertheless, its single-copy join has a two-queue layout in the order

$$4, 0, 7, 6, 2, 1, 3, 5, \tag{7}$$

where $6, 7$ are the apices. The separate certificate in the reproduction appendix includes a verified queue assignment. The counterexample concerns the claimed map, not the NP-hardness conclusion. The present contribution is the deletion lemma and the replication needed for the converse and witness recovery.

The mixed-layout results of de Col, Klute and Nöllenburg [3, Theorems 1–2] have different page types or a prescribed vertex order. Depian et al. [4, Theorems 1–3] develop structural and exact layout algorithms. Their repeated-component arguments depend on bounded structural parameters and do not provide this threshold-raising rule.

Bekos et al. [5, Theorem 1] recently proved that one-queue recognition remains NP-hard for outerplanar graphs of pathwidth at most five. This strengthens the source baseline. Since all outerplanar graphs admit two queues, that result does not itself establish two-queue recognition hardness. Our reduction treats unrestricted source graphs and does not depend on the newer restricted-source theorem.

The literature check through 15 September 2026 found no earlier proof of the exceptional-vertex lemma or this corrected construction in the sources examined. The two-apex idea and the hardness conclusion both have the antecedent in [2]. Priority remains subject to expert assessment, including unindexed corrections and unpublished work. We make no claim that five copies are necessary or that the counterexample itself is new.

References

[1] Lenwood S. Heath and Arnold L. Rosenberg. *Laying Out Graphs Using Queues.* SIAM Journal on Computing 21(5):927–958, 1992. doi:10.1137/0221055.

[2] Philipp de Col. *Algorithms and Drawings for Mixed Linear Layouts of Graphs.* Diploma thesis, TU Wien, 2019. University repository.

[3] Philipp de Col, Fabian Klute, and Martin Nöllenburg. *Mixed Linear Layouts: Complexity, Heuristics, and Experiments.* Graph Drawing and Network Visualization, GD 2019, LNCS 11904, pp. 460–467. arXiv:1908.08938v1.

[4] Thomas Depian, Simon D. Fink, Robert Ganian, and Vaishali Surianarayanan. *Linear Layouts Revisited: Stacks, Queues, and Exact Algorithms.* ESA 2025, LIPIcs 351, 15:1–15:18. doi:10.4230/LIPIcs.ESA.2025.15.

[5] Michael A. Bekos, Thomas Depian, Stefan Felsner, Michael Kaufmann, Philipp Kindermann, Fabrizio Montecchiani, Maria Eleni Pavlidi, Alexandra Weinberger, Alexander Wolff, and Johannes Zink. *On the Recognition of Outerplanar Graphs with Queue Number 1.* Preprint, 27 July 2026. arXiv:2607.24297v1.

[6] Sergey Pupyrev. *Open Problems in Linear Graph Layouts.* Entry "2-QUEUE GRAPH RECOGNITION." Author-maintained agenda, accessed 15 September 2026.

A. Verification and reproduction

The general argument is Lemma 2 and the two witness transformations. The following finite checks provide implementation evidence and targeted attempts to refute that argument. Solver agreement and agent review are not substitutes for the proof or human expert assessment.

The prepared Z3 checker used seven source instances and made fifteen recovery calls. The independently implemented CP-SAT checker used five source instances and made six recovery calls. Together the final suites cover eleven distinct sources, with orders from zero through six and target orders through thirty-two. Their source decisions were cross-checked by exhaustive permutations. Target witnesses were obtained independently of the candidate, then altered by reversal or queue renaming before recovery where specified.

For the six-vertex NO source in Equation 6, Z3 rejected the actual full output. CP-SAT independently rejected a fourteen-vertex induced subgraph of that output. A target layout would restrict to a layout of this subgraph, so this is a valid NO certificate. A separate full-target CP-SAT run was interrupted after the smaller certificate became available; its UNKNOWN result supplies no infeasibility evidence.

An independent reviewer wrote standard-library checks without importing the candidate or the existing checkers. These checked all seventy-six labelled simple source graphs on zero through four vertices and performed 358 recovery calls. The reviewer also tested 16,962 positional hypotheses with four through twelve original vertices, thirteen malformed byte inputs, and two exhaustive checks of 720 orders each: one for the induced K_6 obstruction and one for the single-copy counterexample. Arbitrary-order recovery sampling used the seed recorded in the review script. Finite domains and discarded invalid sampled orders are documented in the raw review record.

The campaign directory is `campaigns/two-queue-recognition`. In its `work` directory, run:

```
python3 check.py --self-test
python3 check.py --candidate algorithm.py
python3 verify.py --candidate algorithm.py
typst compile manuscript.typ manuscript.pdf
```

The implementation requires Python; verification additionally requires Z3 and OR-Tools CP-SAT. Figure generation uses only Python's standard library. The exact versions are in `rounds/001/versions.json`. The self-test includes deliberate incorrect forward and backward subprocess fixtures; both must be rejected. No command imposes a wall-clock timeout.

The forward executable reads `{"adj": A}` and writes the target matrix. Recovery runs in a fresh process:

```
python3 algorithm.py --extract < recovery-input.json
```

Its input object has fields `source` and `target_solution`. A solution has fields `order` and `queues`, with edge labels enumerated lexicographically. The injection harnesses exercise both modes directly; the candidate contains no solver. Malformed sources map to `{"invalid": true}`, a fixed target nonmember.

The final prepared and verification logs are `rounds/001/prepared-after-input-fix.jsonl` and `rounds/001/verification.jsonl`. The subgraph certificate is `rounds/001/induced-no-certificate.json`. The independent review, its executable checks and raw results are in `reviews/native/`. Run its `check.py` from the repository root to reproduce the independent checks. These files also retain the single-copy counterexample and its explicit queue assignment. The research draft has undergone independent agent review; human expert review and priority assessment remain outstanding.